

ORGNZ-99: Best Practice Summary

Series: ORGNZ | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 03/26/2026

Overview

This notebook consolidates every actionable best practice from the ORGNZ series (notebooks 01-10) into a single definitive reference. Each best practice specifies the exact setting, value, or action to take. No hedging — these are the correct choices for organizing data in Dynatrace Grail.

Table of Contents

1. [Bucket Design](#)
 2. [Retention Strategy](#)
 3. [Data Routing](#)
 4. [IAM and Access Control](#)
 5. [Security Context](#)
 6. [Segment Design](#)
 7. [Metadata Enrichment](#)
 8. [Query Performance](#)
 9. [Governance and Operations](#)
-

1. Bucket Design

#	Best Practice	Setting / Value	Priority	Category
1	Keep per-bucket ingest under 1 TB/day	Target: <1 TB/day per bucket; never exceed 3 TB/day	Critical	Performance
2	One data type per bucket	Each bucket stores exactly one table type (logs, metrics, spans, events, bizevents, security_events)	Critical	Architecture
3	Use consistent naming convention	Format: <provider>_<type>_<retention> or <team>_<type>_<retention> (e.g., aws_logs_35d , team_platform_logs)	Critical	Naming
4	Bucket names: lowercase, letter-first, underscores/hyphens only	Regex: ^[a-z][a-z0-9_-]*\$; never use default_* prefix	Critical	Naming
5	Plan bucket names before creation	Bucket names are immutable after creation; display names can be changed	Critical	Planning
6	Stay within the 80-bucket environment limit	Budget buckets across teams; request increase only if justified	Recommended	Planning
7	Create dedicated buckets per cost center	One bucket per business unit/LOB for direct GiB/day cost attribution	Recommended	Cost
8	Create separate buckets for compliance data	Dedicated bucket (e.g., compliance_audit_logs) with extended retention for regulatory requirements	Recommended	Compliance
9	Create short-retention buckets for debug logs	Bucket: debug_logs_7d with 7-day retention for high-volume verbose logs	Recommended	Cost
10	Split high-volume buckets to maintain queryability	If bucket exceeds 1 TB/day, split by team, environment, or application	Recommended	Performance

2. Retention Strategy

#	Best Practice	Setting / Value	Priority	Category
11	Set debug/development log retention to 3-7 days	Bucket retention: 7 days maximum for DEBUG-level logs	Critical	Cost
12	Set standard operational log retention to 35 days	Bucket retention: 35 days for application and infrastructure logs	Recommended	Operations
13	Set performance analysis retention to 60-90 days	Bucket retention: 90 days for metrics and quarterly trend analysis	Recommended	Operations
14	Set audit/compliance log retention to 365-3657 days	Bucket retention: match regulatory requirement (SOX: 7 years = 2,555 days; HIPAA: 6 years = 2,190 days)	Critical	Compliance
15	Set span retention to 10-35 days	Bucket retention: 10 days for default APM; 35 days if long-running incident correlation needed	Recommended	Operations
16	Set security event retention to 90-365 days	Bucket retention: 365 days for security investigation timelines	Recommended	Security
17	Never lower retention without understanding data loss	Reducing retention immediately deletes data older than the new setting; this is irreversible	Critical	Operations

3. Data Routing

#	Best Practice	Setting / Value	Priority	Category
18	Route data to custom buckets via OpenPipeline	Configure OpenPipeline routing rules with conditions on <code>loglevel</code> , <code>host.group</code> , <code>service.name</code> , <code>log.source</code>	Critical	Architecture
19	Always define a default route	Set <code>default: "default_logs"</code> (or appropriate default bucket) as the fallback in every OpenPipeline routing configuration	Critical	Reliability
20	Route DEBUG logs to short-retention buckets	OpenPipeline condition: <code>loglevel == 'DEBUG'</code> → destination: <code>debug_logs_7d</code>	Recommended	Cost
21	Route audit logs to compliance buckets	OpenPipeline condition: <code>log.source contains 'audit'</code> → destination: <code>audit_logs_365d</code>	Recommended	Compliance
22	Data cannot be moved between buckets after ingestion	Route new data via OpenPipeline; old data stays in the original bucket until retention expires	Critical	Architecture

4. IAM and Access Control

#	Best Practice	Setting / Value	Priority	Category
23	Always include <code>storage:buckets:read</code> in every data access policy	Without bucket read, users cannot query any data; always pair with table permissions	Critical	Security
24	Assign policies to groups, never to individual users	Create IAM groups per team/role; assign all policies at the group level	Critical	Governance
25	Use the two-step policy pattern	Step 1: <code>ALLOW storage:buckets:read WHERE ...</code> ; Step 2: <code>ALLOW storage:logs:read</code> (or other table types)	Critical	Security
26	Use <code>STARTSWITH</code> for team bucket prefixes	Policy: <code>ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH "team_platform_"</code> — automatically includes future buckets with matching prefix	Recommended	Scalability
27	Use <code>MATCH</code> operator for array fields	When <code>dt.security_context</code> holds an array, use <code>MATCH</code> (not <code>=</code> , <code>STARTSWITH</code> , or <code>IN</code> — those always return false for arrays)	Critical	Security
28	Start with least privilege	Grant minimum required access; expand only when justified	Critical	Security
29	Use policy boundaries for reusable conditions	Define regional/environmental restrictions once as a boundary; apply to multiple policies	Recommended	Governance
30	Limit policy boundaries to 10 restrictions per boundary	Maximum 10 conditions per boundary; create additional boundaries if more are needed	Recommended	Architecture
31	Use field-level <code>DENY</code> for sensitive data	Policy: <code>DENY storage:logs:read:user.email, storage:logs:read:user.ip_address</code> to mask PII fields	Recommended	Security
32	Test policies with sample users before deployment	Create a test user in the target group and verify query results before rolling out to the team	Critical	Operations
33	Document every IAM policy	Include name, description, tags, target group, and purpose in a policy registry	Recommended	Governance

5. Security Context

#	Best Practice	Setting / Value	Priority	Category
34	Use <code>dt.security_context</code> for record-level access control	Set via OpenPipeline Security Context processor or OneAgent host properties	Critical	Security
35	Use hierarchical encoding in security context values	Format: <code><org>/<department>/<team></code> (e.g., <code>acme/engineering/platform</code>); enables department-level access with <code>MATCH ('acme/engineering/*')</code>	Critical	Scalability
36	Use type prefixes for security context values	Prefix with <code>team:</code> , <code>lob:</code> , <code>env:</code> , <code>cloud:</code> (e.g., <code>team:checkout</code> , <code>lob:finance</code> , <code>env:production</code>)	Recommended	Naming
37	Plan security context hierarchy before implementation	Context values are assigned to records at ingest time; changing schema requires re-ingesting data	Critical	Planning
38	Use security context instead of additional buckets when at 80-bucket limit	Security context enables multi-team isolation within shared buckets without consuming bucket quota	Recommended	Scalability
39	Set security context on entities via Grail Security Context settings	Settings → Topology model → Grail Security Context; entities inherit context to related records	Recommended	Security
40	Always use <code>MATCH</code> for array-valued security context in IAM policies	<code>ALLOW storage:logs:read WHERE storage:dt.security_context MATCH ("team-a*")</code>	Critical	Security

6. Segment Design

#	Best Practice	Setting / Value	Priority	Category
41	Use segments for dynamic data views, not access control	Segments filter at query time; they do not enforce security boundaries — IAM policies do	Critical	Architecture
42	Stay within the 20-include limit per segment	Maximum 20 include blocks per segment; plan rule complexity accordingly	Critical	Architecture
43	One include block per data type	Cannot have two <code>logs</code> includes; combine conditions with <code>OR</code> within a single include	Critical	Architecture
44	Use Primary Grail Fields in segment filters	Filter on <code>dt.host_group.id</code> , <code>k8s.namespace.name</code> , <code>k8s.cluster.name</code> , <code>aws.account.id</code> , <code>azure.subscription</code> , <code>gcp.project.id</code> — these are indexed and propagate across all signal types	Critical	Performance
45	Use prefix-based naming for cross-signal filtering	Name host groups, namespaces, etc. with prefixes (e.g., <code>prod-web-tier</code> not <code>web-tier-prod</code>) so <code>starts-with</code> works on both entity and signal includes	Recommended	Naming
46	Create one segment per dimension	Separate segments for platform, application, environment, and region — users combine them as needed	Recommended	Architecture
47	Add event includes for Davis problem filtering	Define events include with <code>event.kind = "DAVIS_PROBLEM"</code> — entity includes alone do not filter problems	Critical	Troubleshooting
48	Set official segments to Public visibility	Platform team-owned segments: set to Public; application team segments: share with their group	Recommended	Governance
49	Use consistent segment naming prefixes	Prefixes: <code>team-</code> , <code>env-</code> , <code>app-</code> , <code>region-</code> (e.g., <code>team-platform-infra</code> , <code>env-production</code>)	Recommended	Naming
50	Test segment filter conditions with DQL before	Run the equivalent <code>filter</code> clause as a DQL query; verify record counts match expectations	Critical	Operations

#	Best Practice	Setting / Value	Priority	Category
	creating the segment			
51	Limit segment variable dropdowns to 10,000 values	Maximum 10,000 values per variable; 100 selected values per segment	Recommended	Architecture
52	Use entity relationship traversal for comprehensive segments	Top-down: host → process group → service; bottom-up: service → process group → host	Recommended	Architecture
53	Segments reduce scanned bytes	Applying a segment injects filters at query time, reducing data scanned and improving performance on high-volume buckets	Recommended	Performance

7. Metadata Enrichment

#	Best Practice	Setting / Value	Priority	Category
54	Enrich all signals with Primary Grail Fields before building segments	Verify <code>dt.host_group.id</code> , <code>k8s.namespace.name</code> , <code>k8s.cluster.name</code> are present on logs, spans, and metrics	Critical	Architecture
55	Set <code>dt.security_context</code> via host properties for dedicated infrastructure	Deployment Status → select hosts → Modify host properties → set <code>dt.security_context=<team></code>	Recommended	Security
56	Set <code>dt.cost.costcenter</code> and <code>dt.cost.product</code> on hosts	Host properties: <code>dt.cost.costcenter=<center></code> , <code>dt.cost.product=<product></code> — these propagate to service metrics	Recommended	Cost
57	Use <code>DT_TAGS</code> environment variable for shared infrastructure	Set <code>DT_TAGS=primary_tags.team=<team></code> per process when multiple apps share the same host	Recommended	Architecture
58	Restart applications after enrichment changes for spans	Logs: no restart needed (OneAgent auto-enriches); Spans: application restart required; Metrics: OneAgent restart applies automatically	Critical	Operations
59	Verify enrichment coverage before building segments	Run DQL audit query to check field coverage percentage across signal types; fields with 0% coverage need enrichment	Critical	Operations
60	Use Primary Grail Tags (<code>primary_tags.*</code>) when standard fields do not match your organization	Set via host properties or <code>DT_TAGS</code> environment variable; propagates across logs, spans, and topology	Recommended	Architecture

8. Query Performance

#	Best Practice	Setting / Value	Priority	Category
61	Understand the 500 GB scan limit	Maximum 500 GB scanned per query; at 1 TB/day ingest, queryable window is ~12 hours	Critical	Performance
62	Target specific buckets in queries	Use <code>fetch logs</code> , bucket: <code>{"team_logs", "app_logs_*"}</code> to avoid scanning all buckets	Recommended	Performance
63	Use exact match (<code>=</code>) over pattern match in segment filters	<code>=</code> is faster than <code>starts-with</code> ; use exact values when known	Recommended	Performance
64	Minimize segment filter expressions	Fewer conditions = better query performance; maximum 10 expressions per filter	Recommended	Performance
65	Use OpenPipeline pre-processing for complex segment logic	If segment conditions would be complex, pre-enrich a simple field (e.g., <code>dt.security_context</code>) and filter on that	Recommended	Performance
66	Maximum 1,000 records returned per query	Use aggregations (<code>summarize</code> , <code>makeTimeseries</code>) for larger datasets	Recommended	Performance
67	Maximum 10 segments per query	Do not stack more than 10 segments; design broader segments instead of many narrow ones	Recommended	Architecture

9. Governance and Operations

#	Best Practice	Setting / Value	Priority	Category
68	Schedule regular access reviews	Quarterly review of all IAM policies and group memberships; remove unnecessary permissions	Critical	Governance
69	Document all bucket purposes	Maintain a registry mapping each bucket to its owner, data type, retention, cost center, and routing rule	Recommended	Governance
70	Document all security context values	Maintain a registry of all <code>dt.security_context</code> values, their meaning, and which IAM policies reference them	Recommended	Governance
71	Platform team owns infrastructure segments	Platform/SRE team creates and maintains public environment and infrastructure segments	Recommended	Governance
72	Application teams own app-specific segments	Each team creates segments for their applications; share with their IAM group	Recommended	Governance
73	Review segments quarterly	Remove unused segments; verify filter conditions still match data; check for empty results	Recommended	Operations
74	Use combined approach for defense in depth	Buckets for retention/cost + Security context for access control + Segments for dynamic views — all three together	Critical	Architecture
75	Implement in phases	Phase 1: Plan → Phase 2: Buckets + routing → Phase 3: IAM policies → Phase 4: Segments → Phase 5: Governance	Recommended	Operations
76	Selective record deletion is not possible	Cannot delete individual records; wait for bucket retention to expire, or truncate the entire bucket	Critical	Operations
77	Bucket deletion destroys all data permanently	Deleting a bucket removes ALL records immediately and irreversibly	Critical	Operations

This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.